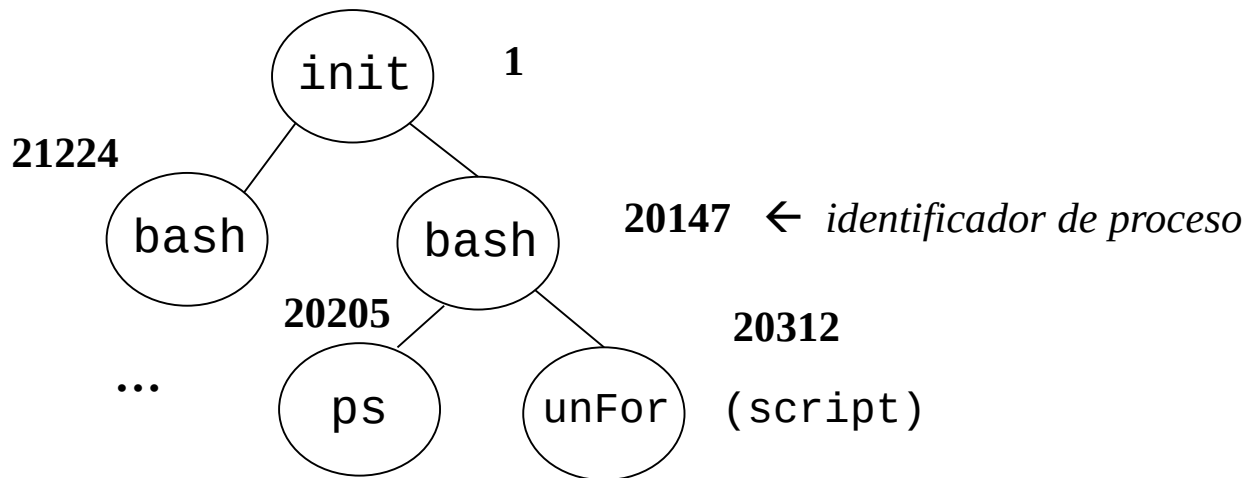


# Procesos

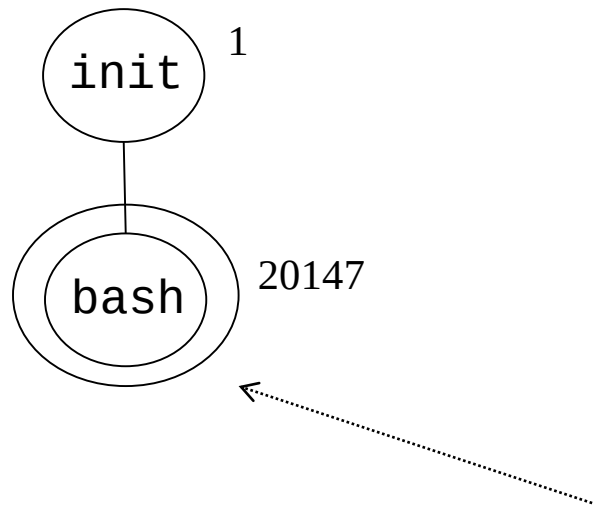
- Proceso: programa en ejecución.
- Programa: conjunto de instrucciones.

Un mismo programa puede ser ejecutado por varios procesos simultáneamente



# Procesos

- Al entrar a la cuenta de un usuario



Este proceso está activo, y puede ejecutar un comando

# ps

- Muestra el estado de los procesos.

**ps** [*opciones*]

- u muestra el usuario que creó el proceso
- a muestra todos los procesos del sistema
- l muestra los procesos en formato largo

**top** *para versiones Linux.*

**ps tree** Indica el árbol de dependencias entre los procesos que se están ejecutando

# ps

**\$ps**

| <b>PID</b>   | <b>TTY</b>   | <b>TIME</b> | <b>CMD</b>  |
|--------------|--------------|-------------|-------------|
| <b>20147</b> | <b>pts/5</b> | <b>0:00</b> | <b>bash</b> |
| <b>20205</b> | <b>pts/5</b> | <b>0:00</b> | <b>ps</b>   |

**pid**    identificador del proceso (**20147**)

**TTY**    terminal asociado al proceso (**/dev/pts/5**)

**TIME**   tiempo de CPU consumido por el proceso (**0:00**)

**CMD**    nombre del comando que crea el proceso (**ps bash**)

# ps

**\$ps -au**

*Estilo BSD*

| USER   | PID   | %CPU | %MEM | VSZ | RSS | TTY  | STAT | START | TIME | COMMAND     |
|--------|-------|------|------|-----|-----|------|------|-------|------|-------------|
| root   | 1     | 0.0  | 3.0  | 44  | 208 | tty1 | S    | 11:10 | 0:00 | init        |
| root   | 43    | 0.0  | 2.3  | 44  | 168 | tty2 | S    | 14:20 | 0:00 | /bin/agetty |
| ...    |       |      |      |     |     |      |      |       |      |             |
| bx1087 | 20150 | 0.1  | 1.3  | 300 | 468 | v04  | S    | 23:10 | 0:01 | -bash       |
| bx1087 | 20412 | 0.1  | 1.3  | 300 | 468 | v04  | S    | 23:10 | 0:01 | ps -au      |

**\$ps -ef**

*Estilo Sistema V*

...

# kill

- Acaba con la ejecución de un proceso

**^C** (control-C) finaliza la ejecución de un proceso interactivo

**kill** [*opciones*] *pid* finaliza la ejecución de un proceso **no** interactivo

| USER   | PID   | %CPU | %MEM | VSZ | RSS | TTY  | STAT | START | TIME | COMMAND     |
|--------|-------|------|------|-----|-----|------|------|-------|------|-------------|
| root   | 1     | 0.0  | 3.0  | 44  | 208 | tty1 | S    | 11:10 | 0:00 | init        |
| root   | 43    | 0.0  | 2.3  | 44  | 168 | tty2 | S    | 14:20 | 0:00 | /bin/agetty |
|        |       | ...  |      |     |     |      |      |       |      |             |
| bx1087 | 20150 | 0.1  | 1.3  | 300 | 468 | v04  | S    | 23:10 | 0:01 | -bash       |
| bx1087 | 20412 | 0.1  | 1.3  | 300 | 468 | v04  | S    | 23:10 | 0:01 | ps -au      |

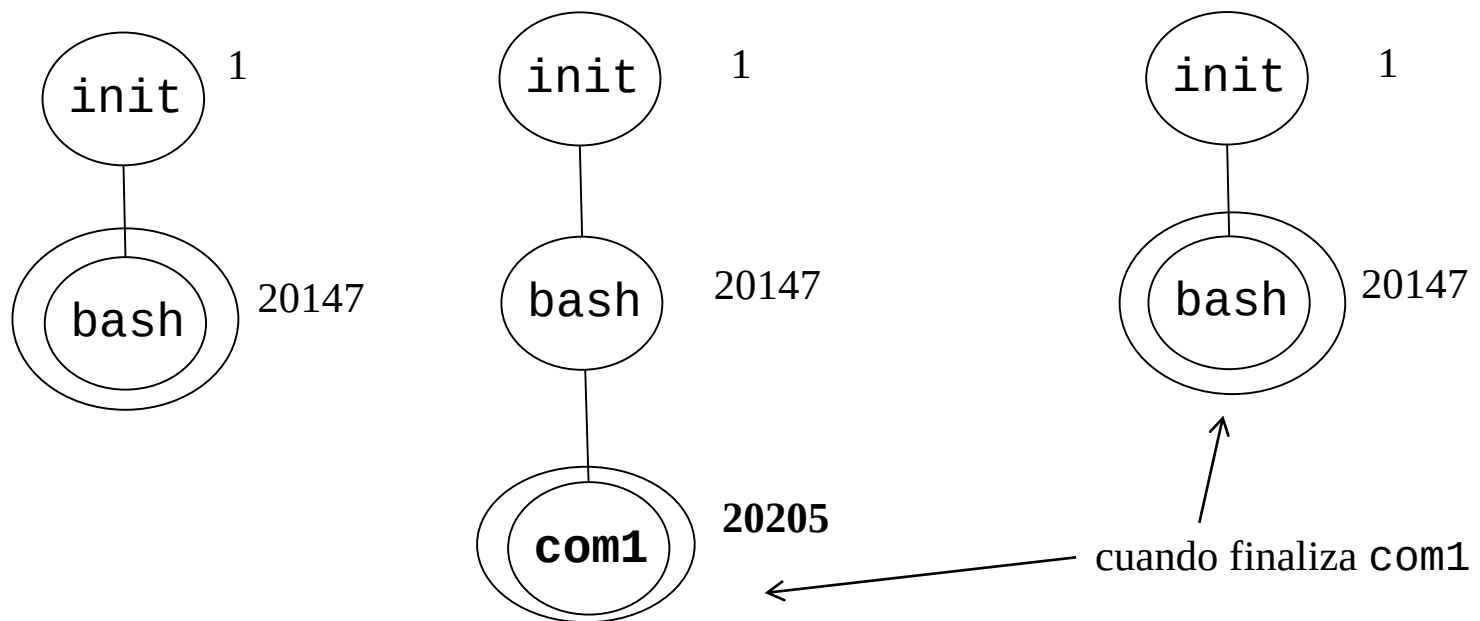
**\$kill 20150**

**\$kill -9 20150**

# Creación de procesos

1. Ejecución en “foreground” o también llamada ejecución interactiva de un comando **com1** (por ejemplo, `ls`, `date`, `who`, `awk`, ... )

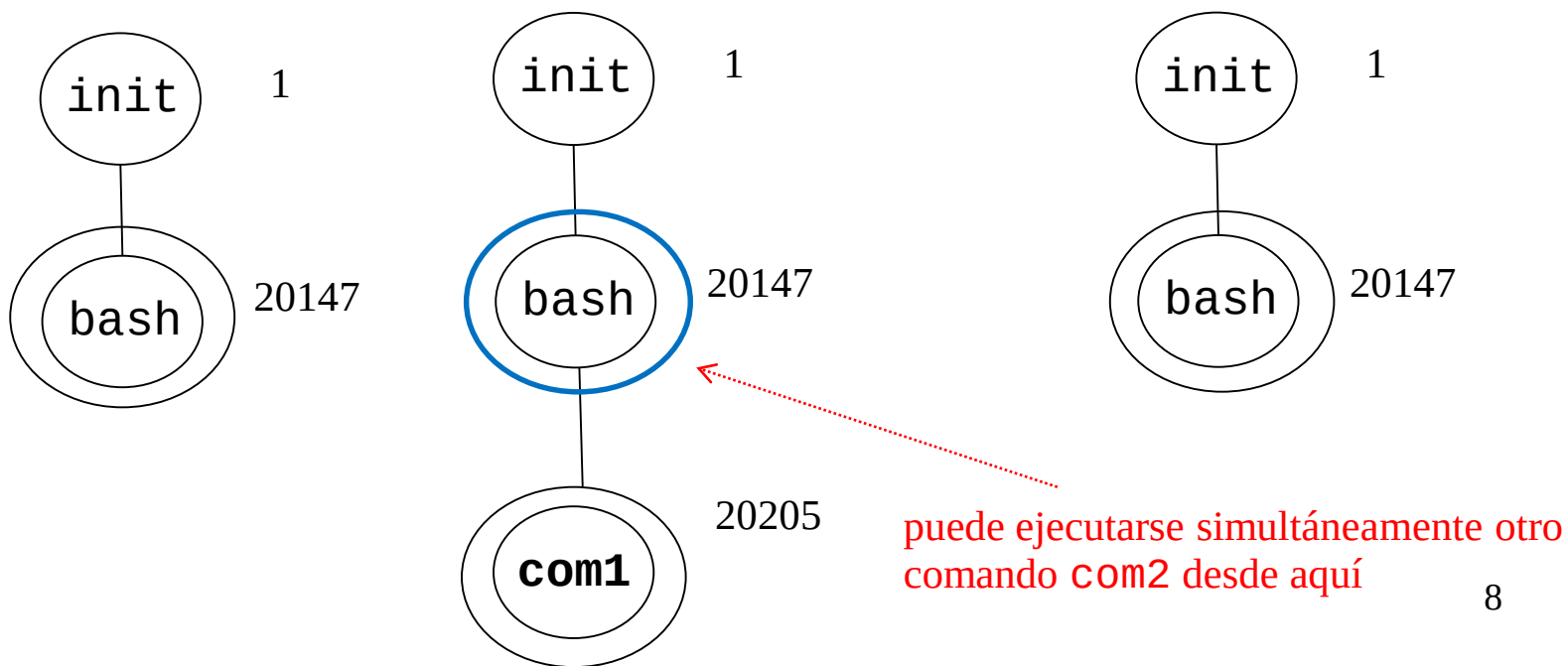
**\$com1**



# Creación de procesos

2. Ejecución en “background” o también llamada ejecución **no** interactiva de un comando **com1** (por ejemplo, **ls**, **date**, **who**, **awk**, **sed**, ...)

**\$com1&**





# Creación de procesos

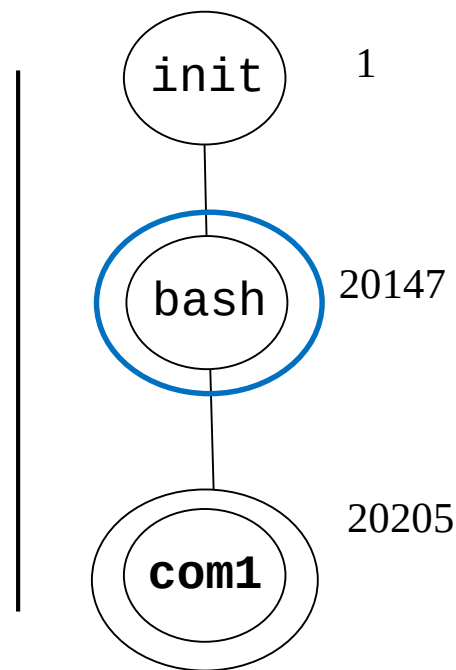
- Al invocar de forma **no** interactiva un comando, la shell informa de su número de proceso y su identificador de tarea o “job”.

**\$com1&**

**[1] 20147**

Con el número de “job” podemos:

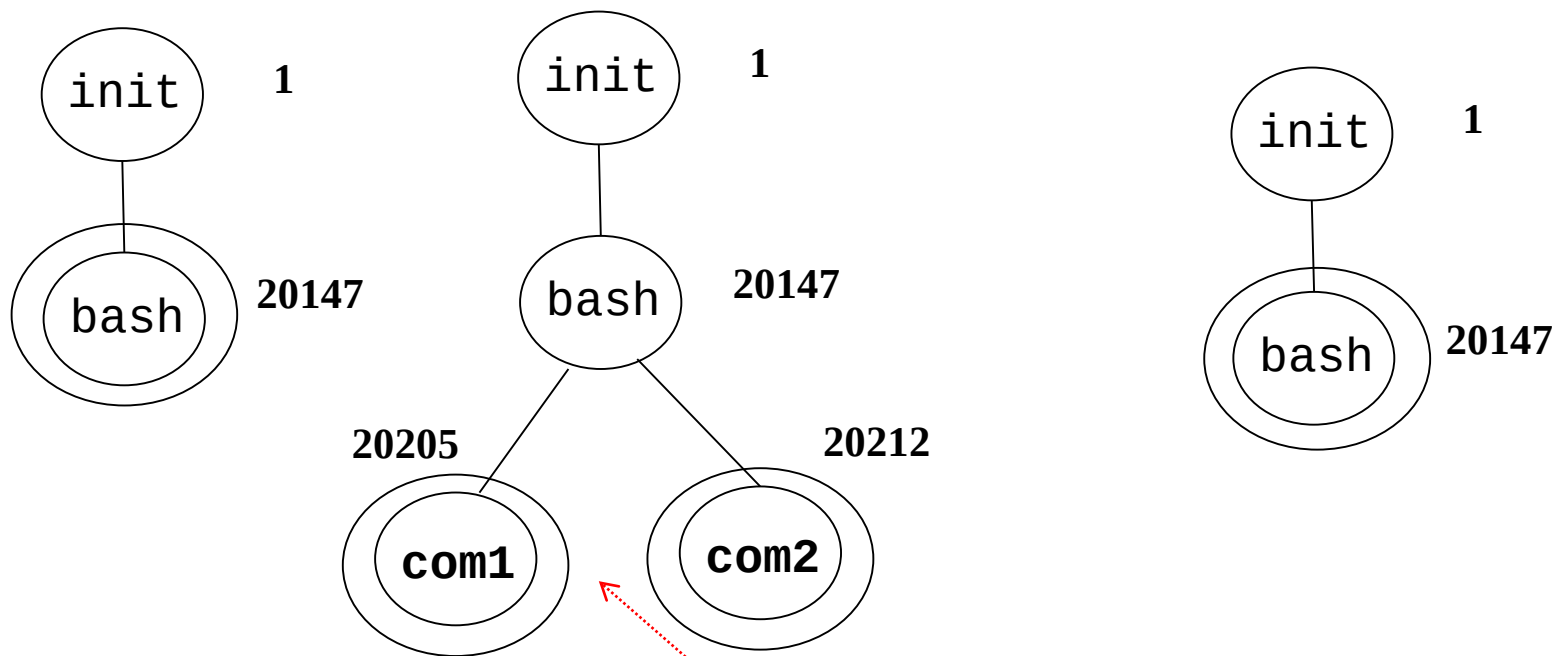
- Terminar el proceso: **kill %1 kill 20147**
- Pausar el proceso: **<CTRL> z**
- Cambiarle de modo a “foreground”: **fg %1**



# Creación de procesos

3. Ejecución a través de una tubería (por ejemplo, `who | wc`)

**\$com1 | com2**

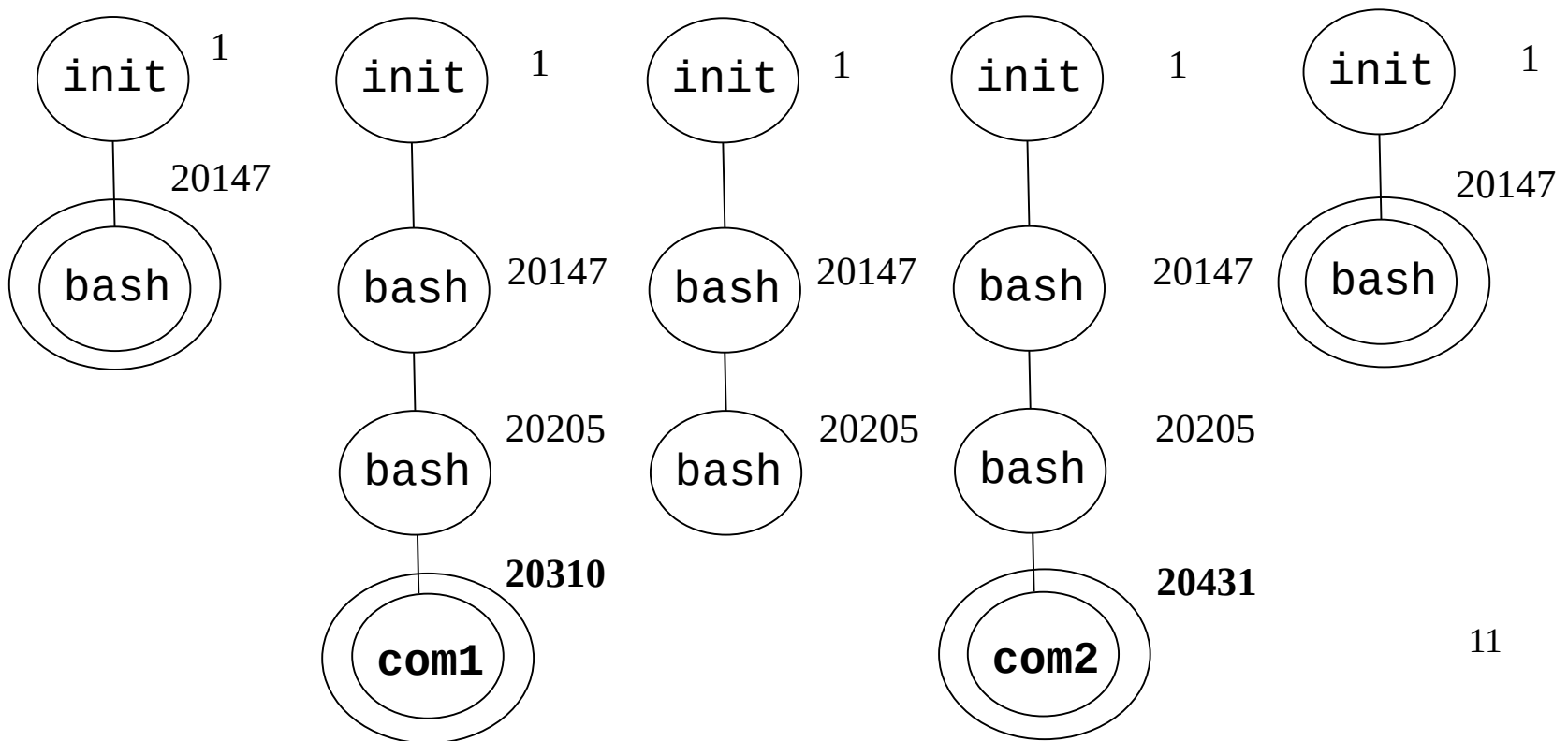


Se crea un tubo para redirigir la entrada/salida

# Creación de procesos

## 4. Ejecución varios comandos entre paréntesis

**`$(com1;com2)`**



# Variables

- El nombre de las variables de la bash pueden contener letras cifras y subrayados ( \_ ).
- La bash distingue entre mayúsculas y minúsculas.
- Para obtener el valor de la variable hay que precederla de **\$**(no confundir con el cursor **\$** de la bash).

```
$numero=4
```

```
$echo "el valor es:" $numero
```

# Variables

- Se puede utilizar `{}` para delimitar el nombre de una variable

```
$num=4
```

```
$num1=33
```

```
$echo "Es ditinto:" ${num}1
```

```
$echo "De esto:" $num1
```

# Variables

- Variables de entorno: al invocar a la bash, se crean automáticamente una serie de variables con unos valores predefinidos:
  - **HOME** indica el directorio de entrada (casa)
  - **PATH** indica la lista de directorios en los que se buscan los comandos o ficheros ejecutables. Cada directorio está separado por :

```
$echo $HOME
```

```
/home/bx1087
```

```
$echo $PATH
```

```
/usr/local/bin:/usr/bin:/bin:/usr/local/games:/usr/games
```

# Variables

- Se puede añadir el directorio actual a la variable PATH para que ejecute ficheros sin necesidad de ./ (por ejemplo, los ejecutables después de compilar en C).

```
$cd ~/tso/01/progs
```

```
$. ./fi00
```

```
$PATH=$PATH:.
```

```
/usr/local/bin:/usr/bin:/bin:/usr/local/games:/usr/games:.
```

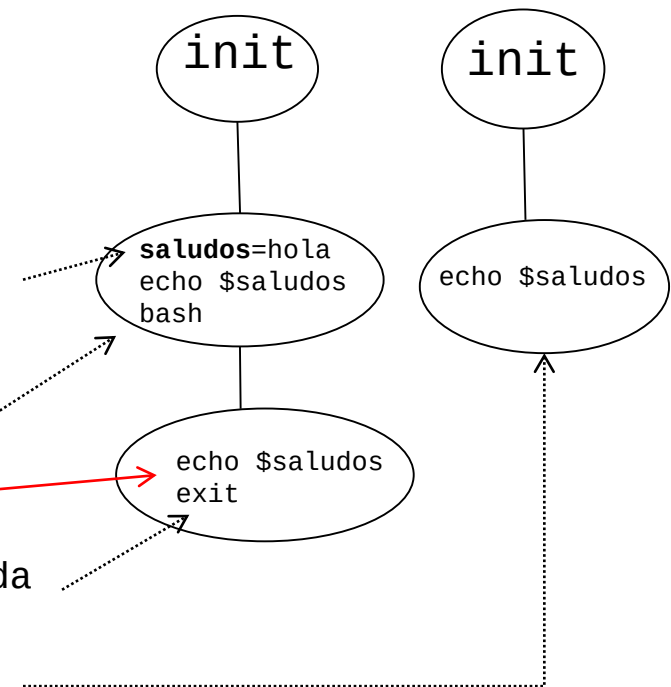
```
$fi00
```

# Variables

- Las variables no predefinidas no se heredan en las bash hijas.

```
$saludos=hola  
$echo $saludos  
$bash  
$echo $saludos  
$exit  
$echo $saludos
```

← dentro bash entrada  
← escribe hola  
← crea bash hija  
← no escribe nada  
← vuelve a bash entrada  
← escribe hola





# Variables

- Se pueden exportar variables a una bash hija (además de las variables predefinidas).

**export** nombVariable

**\$saludos=hola**

← dentro bash entrada

**\$echo \$saludos**

← escribe hola

**\$export saludos**

**\$bash**

← crea bash hija

**\$echo \$saludos**

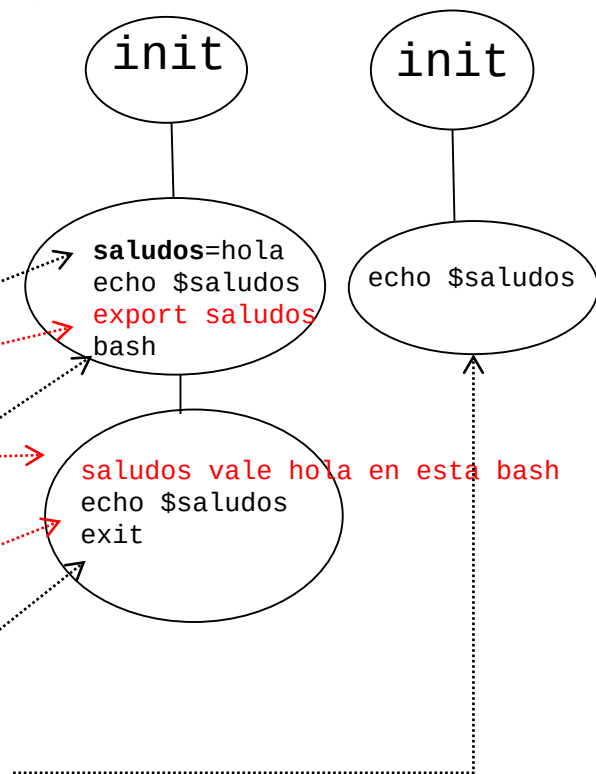
← escribe hola

**\$exit**

← vuelve a bash entrada

**\$echo \$saludos**

← escribe hola



# Variables

- Lo que se exporta es **una copia** de la variable.

**export** nombVariable

**\$saludos=hola**

**\$echo \$saludos**

**\$export saludos**

**\$bash**

**\$echo \$saludos**

**\$saludos=ADIOS**

**\$echo \$saludos**

**\$exit**

**\$echo \$saludos**

← dentro bash entrada

← escribe hola

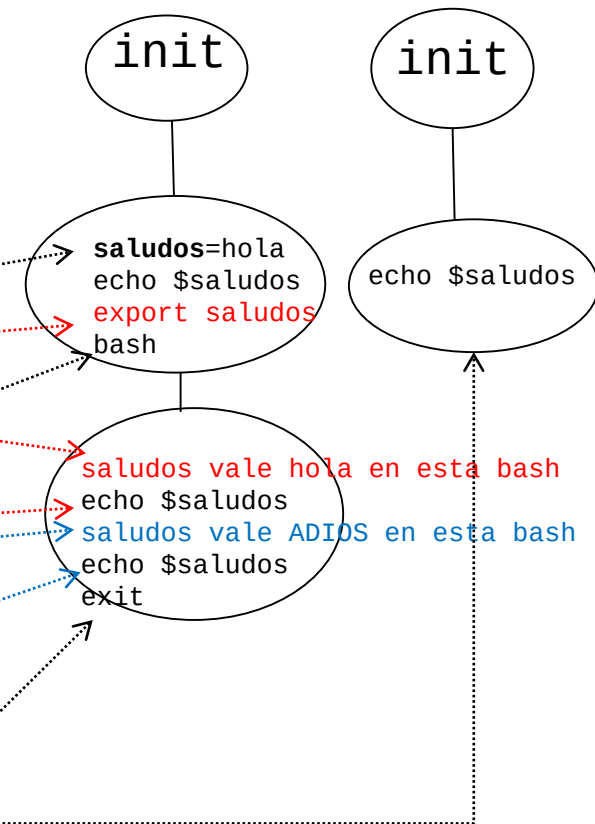
← crea bash hija

← **escribe hola**

← **escribe ADIOS**

← vuelve a bash entrada

← **escribe hola**



# read

- Lee una línea de la entrada estándar y se la asigna a una variable.

**read** nombrVar

- Lee una línea de la entrada estándar y se le asigna un *string* a cada variable.

**read** nombrVar1 nombrVar2 ... nombrVarn

**\$vi compara2numeros**

```
echo "escribe dos numeros:"
read n1 n2
if test $n1 -ge $n2
then
    echo "$n1 es mayor o igual que $n2"
else
    echo "$n1 es menor que $n2"
fi
```

**\$bash** compara2numeros

# Compresión de ficheros

- Compresión de ficheros:

- Genera un fichero con ext. **Z**

**compress** [*opciones*] nombreFich

- Genera un fichero con ext. **gz**

**gzip** [*opciones*] nombreFich

**\$gzip -9 fichero1**

- Descompresión de ficheros:

- Para ficheros con ext. **Z**

**uncompress** [*opciones*] nombreFich

- Para ficheros con ext. **gz**

**gunzip** [*opciones*] nombreFich

**\$gunzip fichero1.gz**

# Compresión de ficheros

- Ver el contenido de ficheros comprimidos (Z y gz):

**zcat** nombreFich

**zless** nombreFich

```
$ls -lR / > fichGrande
```

```
$ls -l fichGrande
```

```
-rw-r--r-- 1 bx1087 bx1087 32813039 nov  5 17:55 fichGrande
```

```
$gzip fichGrande
```

```
$ls -l fichGrande.gz
```

```
-rw-r--r-- 1 bx1087 bx1087 3574366 nov  5 17:55 fichGrande
```

```
$zless fichGrande.gz
```

```
$gunzip fichGrande.gz
```

```
$ls -l fichGrande
```

```
-rw-r--r-- 1 bx1087 bx1087 32813039 nov  5 17:55 fichGrande
```

# Estructuración árbol ficheros

- Creación de un fichero con una estructura conteniendo un árbol de directorios/ficheros (se le suele llamar al resultado fichero tar):

**tar** [*opc.*] *nombreFich.tar nombreOrig*

- c      crea una estructura de directorios y ficheros
- t      examina una estructura de directorios y ficheros
- x      extrae una estructura de directorios y ficheros
- v      se visualiza la operación del tar
- z      realiza la operación tar considerando un fichero comprimido (.gz)

# Estructuración árbol ficheros

- Crea un árbol en un fichero tar (con direccionamiento absoluto)

```
$tar cvf copia_bx1087.tar /home/bx1087
```

```
$tar zcvf copia_bx1087.tgz /home/bx1087
```

- Obtiene de un fichero tar un árbol de directorios/ficheros

```
$tar xvf copia_bx1087.tar
```

```
$tar zxvf copia_bx1087.tgz
```

Debajo del directorio actual **direct** en el que se hace el **tar x...** es de donde colgarán todos los directorios/ficheros que hay en **direct/home/bx1087**

# Estructuración árbol ficheros

- Crea un árbol en un fichero tar (con direccionamiento relativo)

```
$tar cvf copia_bx1087.tar bx1087
```

```
$tar zcvf copia_bx1087.tgz bx1087
```

- Obtiene de un fichero tar un árbol de directorios/ficheros

```
$tar xvf copia_bx1087.tar
```

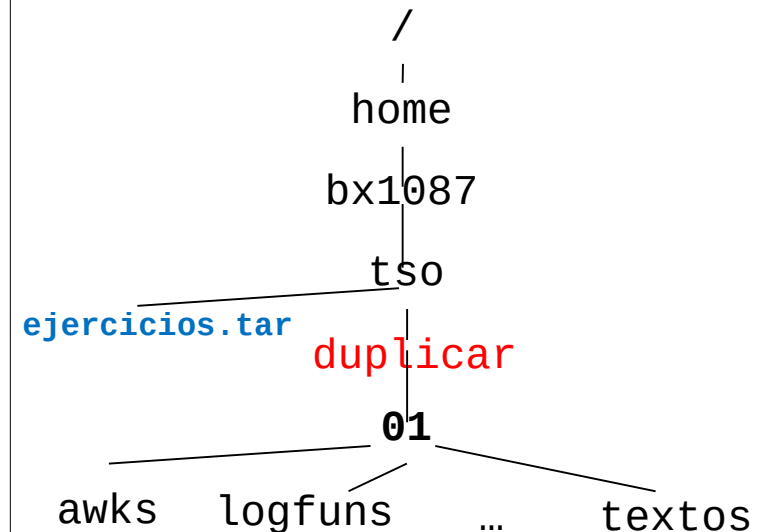
```
$tar zxvf copia_bx1087.tgz
```

Debajo del directorio actual **direct** en el que se hace el **tar x...** es de donde colgarán todos los directorios/ficheros que hay en **direct/bx1087**



# Estructuración árbol ficheros

```
$cd ~/tso/  
$tar cvf ejercicios.tar 01  
  
$mkdir duplicar  
$cd duplicar  
$tar xvf ../ejercicios.tar
```



Debajo del directorio actual `~/tso/duplicar` en el que se hace el **tar x...** es de donde colgarán todos los directorios/ficheros que hay en **duplicar/01**